ACTIVITY 10

GUI with Swing – Part 2 (Take Home)

Name: _____ **Section:** _____

Score Summary		Earned	Possible
Lambda ActionListener + Message Dialogs		X	5
Named Class & Anonymous Class Listeners		X	5
Shared Listener + getActionCommand()		X	5
Input Dialog + MouseAdapter Hover		X	5
Multi-Listener Composition		X	3
Classify: Delegation Model?		X	2
	TOTAL	**XX**	**25**

General Instructions

This activity continues directly from Activity 9. Start from your own Activity 9 submission (same option, same field names). If you no longer have it, rebuild the Part A-D skeleton first, it does not need to be pixel-identical, but it must contain the same required components under the same variable names used below.

Pick the SAME option you used for Activity 9. Every part, and the entire rubric, works exactly the same way no matter which option, only the names differ. The instructions below are written using Option 1 (Task) as the running worked example; if you chose Option 2 or Option 3 for Activity 9, keep using that option's names here, substituting with Activity 9's glossary table.

Part A. Lambda ActionListener + Message Dialogs

Replace Activity 9's simple print-only ActionListener on addButton (“Add Task”) with real validation feedback, using a lambda:

- If Task Name or Due Date is empty (after trimming), show JOptionPane.showMessageDialog(...) with JOptionPane.ERROR_MESSAGE, and return without changing anything else.
- Otherwise, add the entered task to your visible task list (convert historyList's backing array into a DefaultListModel<String> if you have not already, so items can be added at runtime), then show a second showMessageDialog(...) with JOptionPane.INFORMATION_MESSAGE confirming the task was added.

- Required Screenshot 1 (“Empty-Field Error”): leave Task Name or Due Date empty, click Add Task, and capture the ERROR_MESSAGE dialog on screen. Add it as Slide 1 of Screenshots.pptx.
- Required Screenshot 2 (“Task Added”): fill in a real Task Name and Due Date, click Add Task, and capture the INFORMATION_MESSAGE dialog together with the task list already showing the new entry underneath. Add it as Slide 2.

Part B. Named Class & Anonymous Class Listeners

Add two new buttons to your button row: “Clear Form” and “Cancel”. Each must use a DIFFERENT listener style from the module, and different from the lambda you just used in Part A.

- clearButton: create a SEPARATE, NAMED top-level class (its own .java file, e.g. ClearFormListener.java) that implements ActionListener, and resets every input field to its default (blank text fields, unchecked checkboxes, the default radio button reselected). Register an instance of it on clearButton.
- cancelButton: register an ANONYMOUS INNER CLASS implementing ActionListener directly on cancelButton. Inside it, call showConfirmDialog(...) asking “Discard everything currently typed into the form?” with YES_NO_OPTION. If the user chooses YES_OPTION, clear the text fields; otherwise, leave everything as it is.
- Required Screenshot 3 (“Confirm Cancel”): type something into the form, click Cancel, and capture the showConfirmDialog on screen. Add it as Slide 3 of Screenshots.pptx.

Part C. Shared Listener + getActionCommand()

Add two more buttons, “Mark Low” and “Mark High”.

- Create exactly ONE ActionListener (any style) and register that SAME listener object on both buttons, exactly like the module's shared saveButton/cancelButton example.
- Inside it, use e.getActionCommand() to tell which button was actually clicked, and set priorityBox's selection to "Low" or "High" accordingly.

Part D. Input Dialog + MouseAdapter Hover

- Add a “Quick Add” button. Its ActionListener calls JOptionPane.showInputDialog(...) asking for a quick task name. If the result is null (Cancel was clicked), do nothing harmful, just skip adding anything. If it is a non-blank String, add it to your visible task list.
- Register a MouseListener on addButton (“Add Task”) using a MouseAdapter, overriding ONLY mouseEntered and mouseExited: on enter, change addButton's background color; on exit, change it back.
- Required Screenshot 4 (“Quick Add Dialog”): click Quick Add and capture the showInputDialog on screen. Add it as Slide 4 of Screenshots.pptx.

- Required Screenshot 5 (“Hover Effect”): hover your mouse over Add Task and capture the window while its background color is changed. Add it as Slide 5. Your Screenshots.pptx should now have exactly 5 slides.

Part E. Multi-Listener Composition

Confirm that addButton now has BOTH an ActionListener (from Part A, the actual add-task logic) and a MouseListener (from Part D, the hover effect) registered on it at the same time, each one doing its own job with no knowledge that the other exists, exactly like the module's upgraded login form example. No new code is required for this part beyond what Parts A and D already built, but review both listeners together before submitting: clicking must still validate and add correctly, and hovering must still change color, on the very same button. Screenshot 5 from Part D already serves as this part's proof, no additional slide is required.

Part F. Classify: Delegation Model

Answer both questions in 1-2 sentences each. Add your answers as a multi-line comment at the bottom of your tester/main class, not as a separate file.

- Why can ActionListener be written as a one-line lambda, but panel.addMouseListener(e -> ...) fails to compile?
- In Part E, registering an ActionListener and a MouseListener on the same button did not require either one to know the other exists. Which idea from this module does that demonstrate, and why does it matter for building larger GUIs?

Submission Instructions

- Submit your complete working project folder exactly as your IDE created it (NetBeans, Eclipse, IntelliJ IDEA, or similar), including its build and configuration files and your ClearFormListener.java (or your option's equivalent named class file), not just the loose .java files by themselves. Your instructor must be able to open the project folder and run it directly.
- Submit exactly one Screenshots.pptx file containing your 5 required screenshots, one per slide, in order, each slide titled with its checkpoint name (Slide 1: Empty-Field Error, Slide 2: Task Added, Slide 3: Confirm Cancel, Slide 4: Quick Add Dialog, Slide 5: Hover Effect).
- Every slide's screenshot must show your own full name and section in the title bar, exactly as enrolled, the same requirement as Activity 9. A slide that does not, or that appears to be a stock or AI-generated image rather than a real running program, earns zero for that part regardless of code correctness.
- Programs must compile and run without errors. Non-compiling code can still earn partial credit for correct logic, at the instructor's discretion.
- Follow standard Java naming conventions: camelCase for variables and methods, PascalCase for class names.

- State clearly at the top of your submission which option (1, 2, or 3) you chose. It must match the option you used in Activity 9.
- Do not submit your Part F written answers as a separate file. Add them as a multi-line comment at the bottom of your main class instead.

Required File Header

Every .java file you submit must begin with this exact multi-line comment, filled in with your own information:

```
/*
 * IT 203 - Activity 10: GUI with Swing II
 * Name: <Your Full Name>
 * Section: <Your Section>
 * Option Chosen: <1, 2, or 3, matching Activity 9>
 * Date: <Submission Date>
 */
```

Packaging and Submission

Place your complete project folder (copied directly from your IDE, not reorganized by hand) and your Screenshots.pptx file side by side, then compress both together into a single zip file named Section_LastName.zip in camel case, for example BSIT2H_DeJesus.zip.

Submissions that do not follow these instructions may not be checked and will be returned for revision. Late submissions and resubmissions past the approved deadline will receive a point deduction.

SCORING RUBRIC

GUI with Swing – Part 2 (Take Home)

Criterion	Points	Full Credit	Deductions
Part A - Lambda ActionListener + Message Dialogs (5 points)			
Empty-field validation shows the correct dialog type	2	Empty Task Name or Due Date shows ERROR_MESSAGE and stops; a valid entry shows INFORMATION_MESSAGE confirming the add.	**1 pt:** dialog shown but wrong message type, or shown even when validation should have stopped it. **0 pts:** no validation, or no dialogs at all.
Successful add updates the visible task list	1	The task list (JList backed by a DefaultListModel) gains the new entry immediately after a valid Add Task click.	**0 pts:** list does not update, or a static array is still used instead of a DefaultListModel.
Screenshots 1-2 provided and match a real run	2	Slide 1 shows the ERROR_MESSAGE dialog; Slide 2 shows the INFORMATION_MESSAGE dialog together with the updated task list, both with the correct title bar.	**1 pt:** only one of the two slides present or matching. **0 pts:** both missing, mismatched title, or not a real screenshot.
Part B - Named Class & Anonymous Class Listeners (5 points)			
Clear Form uses a separate, named ActionListener class	2	A top-level class implementing ActionListener, in its own file, correctly resets every field, registered on clearButton.	**1 pt:** logic correct but implemented as a lambda or anonymous class instead of a named class. **0 pts:** missing, or does not compile.
Cancel uses an anonymous inner class with showConfirmDialog	2	An anonymous ActionListener on cancelButton calls showConfirmDialog with YES_NO_OPTION and correctly branches on YES_OPTION vs. the other result.	**1 pt:** dialog shown but the Yes/No branches are not both handled correctly. **0 pts:** missing, or implemented as a lambda/named class instead of anonymous.
Screenshot 3 provided and matches a real run	1	Slide 3 shows the showConfirmDialog on screen, correct title bar.	**0 pts:** missing, mismatched title, or not a real screenshot.

officeofthedean.cict@bulsu.edu.ph
(044) 919 7800 Local 1102
McArthur Highway, City of Malolos 3000, Bulacan, Philippines

Criterion	Points	Full Credit	Deductions
Part C - Shared Listener + getActionCommand() (5 points)			
One shared listener object registered on both buttons	2	Exactly one ActionListener instance is created and added to both Mark Low and Mark High, not two separate listener objects.	**0 pts:** two separate listeners used, defeating the point of this part.
getActionCommand() correctly tells the two buttons apart	3	The listener reads e.getActionCommand() and sets priorityBox to "Low" or "High" correctly for each button.	**1-2 pts:** logic present but does not use getActionCommand() (e.g., hardcodes one branch), or sets the wrong value. **0 pts:** missing or does not compile.
Part D - Input Dialog + MouseAdapter Hover (5 points)			
Quick Add uses showInputDialog and handles both outcomes	1	A non-blank answer is added to the task list; a canceled dialog (null) is handled without adding anything or crashing.	**0 pts:** neither outcome handled, or missing.
MouseAdapter overrides only mouseEntered/mouseExited	1	Uses MouseAdapter (not the full MouseListener interface) and overrides exactly these two methods.	**0 pts:** implements the full MouseListener interface with empty bodies instead of MouseAdapter, or missing.
Hover effect visibly changes and reverts	1	addButton's appearance visibly changes on mouseEntered and returns to normal on mouseExited.	**0 pts:** no visible change, or does not revert.
Screenshots 4-5 provided and match a real run	2	Slide 4 shows the showInputDialog on screen; Slide 5 shows addButton with its hover color applied, both with the correct title bar.	**1 pt:** only one of the two slides present or matching. **0 pts:** both missing, mismatched title, or not a real screenshot.
Part E - Multi-Listener Composition (3 points)			
Both listeners coexist correctly on addButton	3	addButton's ActionListener (Part A) and MouseListener (Part D) both remain fully functional at once, neither one interferes with or replaces the other.	**1-2 pts:** one of the two listeners was accidentally removed or broken while adding the other. **0 pts:** only one listener present.
Part F - Classify: Delegation Model? (2 points)			
Both questions answered with real reasoning	2	Correctly explains that ActionListener is a functional interface (one abstract method) so a lambda can implement it, while MouseListener declares five methods so Java cannot infer which one a lambda body	**1 pt:** one question missing or unjustified. **0 pts:** both missing or unjustified.

Criterion	Points	Full Credit	Deductions
		would be; and identifies the Observer pattern / listener independence as why multiple listeners can coexist without coupling.	